

Linux Administration Assessment

SECTION A — CONCEPT APPLICATION

S1. Linux History, Distributions, Server Usage and CLI

Answer

1. History and Evolution of Linux

Linux is an open-source operating system kernel created by Linus Torvalds in 1991. It was initially developed as a free alternative for Unix-like systems. Over time, Linux became widely used because its source code is available, it is customizable, secure, and supported by a large community.

Different organizations developed Linux distributions by combining the Linux kernel with system utilities, package managers and applications. Today Linux is commonly used in servers, cloud computing, networking, cybersecurity, embedded systems and supercomputers.

2. Difference between Ubuntu, CentOS, Debian and RHEL

Distribution Main Feature		Package Manager	Common Use
Ubuntu	User-friendly and popular	APT	Desktop, cloud and servers
Debian	Stable and community-driven	APT	Servers and general use
CentOS	Enterprise-oriented and RHEL-compatible	DNF/YUM	Servers and learning
RHEL	Commercial enterprise Linux	DNF/YUM	Enterprise servers

Note: CentOS Linux 8 and earlier are discontinued; CentOS Stream is the current CentOS project. RHEL is commercially supported by Red Hat.

3. Why Linux is preferred for servers

Linux is widely used in server environments because:

- It is open-source and highly customizable.
- It provides good security and stability.
- It can run for long periods without frequent rebooting.
- It supports powerful command-line administration.
- It works efficiently with server hardware.
- It has strong networking and security tools.
- It is widely supported in cloud and enterprise environments.

4. CLI vs GUI

The CLI (Command Line Interface) is generally more appropriate for servers because administrators can manage the system remotely using SSH, execute commands quickly, automate tasks using scripts and use fewer system resources.

A GUI can be useful for beginners or tasks that are easier to perform visually, but it consumes additional memory and CPU resources.

Conclusion: For production servers, CLI is commonly preferred because it is lightweight, fast, remotely accessible and easy to automate.

S2. Linux File and Directory Commands

The basic Linux commands used for file and directory management are:

Command	Purpose	Example
ls	Lists files and directories	ls -l /etc
cd	Changes the current directory	cd /etc
pwd	Displays current directory	pwd
mkdir	Creates a directory	mkdir /testdir
rm	Removes files/directories	rm test.txt
cp	Copies files/directories	cp file1.txt /testdir/
mv	Moves or renames files/directories	mv old.txt new.txt

Real server management examples

1. ls

ls

ls -l

ls -l /etc

Used to view files, permissions, ownership and directory contents.

2. cd

cd /etc

Used to move between directories.

3. pwd

pwd

Used to check the current working directory.

4. mkdir

```
mkdir /testdir
```

Used to create directories for storing files or application data.

5. rm

```
rm test.txt
```

Used to remove unwanted files.

6. cp

```
cp config.txt /backup/
```

Used to create a copy or backup of files.

7. mv

```
mv oldname.txt newname.txt
```

Used to rename or move files.

These commands are basic but very important for Linux server administration.

S3. Linux Server Installation and Initial Configuration

Answer

The following steps can be used to install Ubuntu Server, CentOS Stream or RHEL in VMware/VirtualBox.

Step 1: Create the Virtual Machine

1. Open VMware/VirtualBox.
2. Select New Virtual Machine.
3. Select the Linux ISO image.
4. Allocate suitable RAM and CPU.
5. Create a virtual hard disk.
6. Start the virtual machine.

Step 2: Install Linux

Follow the installation wizard:

- Select language.
- Select keyboard layout.
- Configure storage.
- Set timezone.
- Create an administrator/root account.

- Install the operating system.
- Reboot the VM after installation.

Step 3: Configure hostname

Set the hostname:

```
hostnamectl set-hostname alphatech-server
```

Verify:

```
hostnamectl
```

The hostname helps identify the server on the network and makes administration easier.

Step 4: Configure a static IP

A server should normally have a stable IP address so that clients can reliably connect to services such as SSH, DNS and web applications.

On RHEL/CentOS:

```
nmcli connection show
```

Example:

```
nmcli connection modify "System eth0" ipv4.addresses 192.168.1.100/24
```

```
nmcli connection modify "System eth0" ipv4.gateway 192.168.1.1
```

```
nmcli connection modify "System eth0" ipv4.dns 8.8.8.8
```

```
nmcli connection modify "System eth0" ipv4.method manual
```

```
nmcli connection up "System eth0"
```

Important: The connection name may be different on the VM. First check it using:

```
nmcli connection show
```

Step 5: Enable SSH

Install OpenSSH if required:

```
dnf install openssh-server -y
```

Start and enable it:

```
systemctl enable --now sshd
```

Check:

```
systemctl status sshd
```

Step 6: Initial verification

Check IP:

`ip addr`

Check routing:

`ip route`

Test connectivity:

`ping 8.8.8.8`

Test DNS:

`ping google.com`

Why these configurations are important

- Hostname: Identifies the server.
- Static IP: Provides a stable address for clients and services.
- SSH: Allows remote administration.
- System verification: Confirms that network, hostname and services are working before the server is placed into production.

S4. Linux Users and Groups

Answer

Linux is a multi-user operating system. Different users can have separate accounts, files and permissions.

Users

A user account represents a person or service that can access the Linux system.

Example:

`useradd dev1`

Groups

A group is a collection of users. Groups make it easier to assign permissions to multiple users.

Example:

`groupadd devteam`

User permissions vs Group permissions

Linux permissions are divided into:

Owner Group Others

rwX rwX rwX

- Owner: Permissions given to the file owner.

- Group: Permissions given to members of the file's group.
- Others: Permissions given to all other users.

useradd

Creates a new user:

useradd dev1

groupadd

Creates a new group:

groupadd devteam

passwd

Sets or changes a user's password:

passwd dev1

Add user to group

usermod -aG devteam dev1

User isolation

User isolation is important because each employee should only have access to the files and resources required for their work. It helps prevent accidental modification, unauthorized access and data loss.

S5. Linux Directory Structure

Answer

Linux uses a hierarchical directory structure. Important directories include:

/

The root directory is the top-level directory of the Linux filesystem. All other directories exist below /.

Example:

/

├── etc

├── var

├── home

└── ...

/etc

Contains system-wide configuration files.

Examples:

/etc/ssh/

/etc/hosts

/etc/passwd

/var

Contains variable data that changes during system operation.

Examples include:

- Logs
- Spool files
- Cache
- Application data

For example:

/var/log/

contains system logs.

/home

Contains personal directories of normal users.

Example:

/home/dev1

/home/dev2

Why Linux separates directories

The separation makes the system easier to manage, secure and maintain.

Configuration files, user data, logs and system files are stored in different locations.

This also makes backup, troubleshooting and permission management easier on servers.

S6. LVM — Logical Volume Management

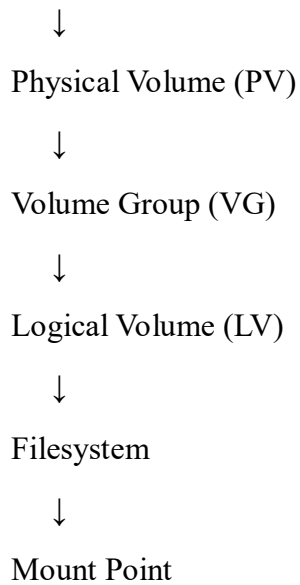
Answer

LVM (Logical Volume Management) is a Linux storage management system that provides flexible management of disk space.

It allows administrators to increase or decrease logical storage without depending only on traditional fixed partitions.

Main Components

Physical Disk



1. Physical Volume — PV

A physical disk or partition prepared for LVM.

Example:

```
pvccreate /dev/sdb
```

2. Volume Group — VG

A storage pool created from one or more physical volumes.

```
vgcreate alphavg /dev/sdb
```

3. Logical Volume — LV

A virtual storage volume created from the volume group.

```
lvcreate -L 5G -n datalv alphavg
```

Advantages of LVM

- Storage can be extended easily.
- Multiple disks can be combined.
- Logical volumes are easier to manage.
- Storage can be resized according to requirements.
- It provides flexibility compared with fixed partitions.

Production use cases

LVM is commonly used for:

- Database servers
- File servers

- Application servers
- Virtualization servers
- Servers requiring storage expansion

SECTION B — PRACTICAL SERVER ADMINISTRATION

Assumption: The practical below uses RHEL 9 / CentOS Stream. Run commands as root or use sudo.

TASK 1 — Linux Installation, Hostname & Core Commands

1. Install Linux

Create a VM in VMware/VirtualBox and install RHEL/CentOS from the ISO.

After installation, log in as root.

Check the OS:

```
cat /etc/os-release
```

2. Set hostname

```
hostnamectl set-hostname alphatech-server
```

Verify:

```
hostnamectl
```

Expected:

```
Static hostname: alphatech-server
```

3. Configure Static IP

First find the connection name:

```
nmcli connection show
```

Example:

```
NAME
```

```
System eth0
```

Configure it:

```
nmcli connection modify "System eth0" ipv4.addresses 192.168.1.100/24
```

```
nmcli connection modify "System eth0" ipv4.gateway 192.168.1.1
```

```
nmcli connection modify "System eth0" ipv4.dns 8.8.8.8
```

```
nmcli connection modify "System eth0" ipv4.method manual
```

```
nmcli connection up "System eth0"
```

Replace the IP, gateway and connection name according to your actual network.

Verify:

`ip addr`

`ip route`

4. Enable SSH

Install SSH server:

`dnf install openssh-server -y`

Enable and start:

`systemctl enable --now sshd`

Verify:

`systemctl status sshd`

You should see:

Active: active (running)

5. Verify Connectivity

Test gateway:

`ping -c 4 192.168.1.1`

Test Internet:

`ping -c 4 8.8.8.8`

Test DNS:

`ping -c 4 google.com`

6. Practice Core Commands

List /etc

`ls -l /etc`

Create /testdir

`mkdir /testdir`

Verify:

`ls -ld /testdir`

Create a test file

`touch /testdir/test.txt`

Check:

`ls -l /testdir`

Copy the file

```
cp /testdir/test.txt /testdir/test-copy.txt
```

Verify:

```
ls -l /testdir
```

Move/rename the file

```
mv /testdir/test-copy.txt /testdir/renamed.txt
```

Verify:

```
ls -l /testdir
```

Delete the file

```
rm /testdir/renamed.txt
```

Verify:

```
ls -l /testdir
```

Final verification

```
hostnamectl
```

```
ip addr
```

```
ip route
```

```
systemctl status sshd
```

```
ls -l /etc
```

```
ls -l /testdir
```

TASK 2 — User & Group Management

1. Create devteam group

```
groupadd devteam
```

Verify:

```
getent group devteam
```

2. Create dev1

```
useradd dev1
```

Set password:

```
passwd dev1
```

Enter the password when prompted.

3. Create dev2

```
useradd dev2
```

Set password:

```
passwd dev2
```

4. Add users to devteam

```
usermod -aG devteam dev1
```

```
usermod -aG devteam dev2
```

5. Verify group membership

For dev1:

```
id dev1
```

For dev2:

```
id dev2
```

Expected output should contain:

```
groups=...,devteam
```

You can also use:

```
groups dev1
```

```
groups dev2
```

Final verification

```
getent group devteam
```

```
id dev1
```

```
id dev2
```

TASK 3 — File System Permissions & Network Configuration

1. Create project directories

```
mkdir -p /project/docs
```

```
mkdir -p /project/scripts
```

Verify:

```
ls -l /project
```

2. Set ownership

The requirement is:

Owner = dev1

Group = devteam

Use:

```
chown -R dev1:devteam /project
```

Verify:

```
ls -ld /project
```

```
ls -ld /project/docs
```

```
ls -ld /project/scripts
```

3. Set permissions

Requirement:

Owner → rwx

Group → r-x

Others → ---

This is:

750

Apply:

```
chmod 750 /project
```

```
chmod 750 /project/docs
```

```
chmod 750 /project/scripts
```

Verify:

```
ls -ld /project
```

```
ls -ld /project/docs
```

```
ls -ld /project/scripts
```

Expected permission pattern:

drwxr-x---

4. Create a symbolic link

Create a test file:

```
touch /project/docs/document.txt
```

Create symbolic link:

```
ln -s /project/docs/document.txt /project/docs/document-softlink
```

Verify:

```
ls -l /project/docs
```

The symbolic link should show something similar to:

```
document-softlink -> /project/docs/document.txt
```

5. Create a hard link

```
ln /project/docs/document.txt /project/docs/document-hardlink
```

Verify:

```
ls -li /project/docs
```

The original file and hard link should have the same inode number.

6. Configure Network

Check network interface:

```
ip addr
```

Check NetworkManager connections:

```
nmcli connection show
```

Configure static IP:

```
nmcli connection modify "System eth0" ipv4.addresses 192.168.1.100/24
```

```
nmcli connection modify "System eth0" ipv4.gateway 192.168.1.1
```

```
nmcli connection modify "System eth0" ipv4.dns 8.8.8.8
```

```
nmcli connection modify "System eth0" ipv4.method manual
```

```
nmcli connection up "System eth0"
```

Again, use your actual connection name/IP/gateway.

7. Verify Network

IP address

```
ip addr
```

Routing table

```
ip route
```

Test Internet

```
ping -c 4 google.com
```

8. Verify Hostname

```
hostnamectl
```

Expected:

Static hostname: alphatech-server

TASK 4 — LVM Storage Configuration & System Verification

Important: For this task, use a new/empty virtual disk, such as /dev/sdb.

Do not run pvcreate on your OS disk such as /dev/sda unless you are absolutely sure it is the intended disk.

1. Check available disks

lsblk

Example:

NAME	SIZE	TYPE
------	------	------

sda	20G	disk
-----	-----	------

└─sda1	20G	part
--------	-----	------

sdb	10G	disk
-----	-----	------

Here, /dev/sdb is the additional disk.

2. Create Physical Volume

pvcreate /dev/sdb

Verify:

pvs

3. Create Volume Group

vgcreate alphavg /dev/sdb

Verify:

Vgs

4. Create Logical Volume

For example, create a 5 GB LV:

lvcreate -L 5G -n data1v alphavg

Verify:

lvs

5. Format the Logical Volume

Using XFS:

mkfs.xfs /dev/alphavg/data1v

Or using ext4:

mkfs.ext4 /dev/alphavg/data1v

Use only one of the above.

6. Create /data

```
mkdir /data
```

7. Mount the Logical Volume

```
mount /dev/alphavg/data1v /data
```

Verify:

```
df -h /data
```

You should see the mounted logical volume.

8. Make the Mount Persistent

Find the UUID:

```
blkid /dev/alphavg/data1v
```

Example:

```
UUID="xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx"
```

Edit /etc/fstab:

```
vi /etc/fstab
```

For XFS, add:

```
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx /data xfs defaults 0 0
```

For ext4:

```
UUID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxx /data ext4 defaults 0 0
```

Save and test:

```
mount -a
```

If there is no error, verify:

```
df -h
```

9. Extend the Logical Volume

First check available space:

```
vgs
```

If the volume group has free space, extend the LV.

For example, add 2 GB:

```
lvextend -L +2G /dev/alphavg/data1v
```

For XFS, grow the filesystem:

xfs_growfs /data

For ext4:

resize2fs /dev/alphavg/data1v

Verify:

df -h /data

10. Final System Verification

Verify users

id dev1

id dev2

Verify group

getent group devteam

Verify network

ip addr

ip route

ping -c 4 google.com

Verify hostname

hostnamectl

Verify SSH

systemctl status sshd

Verify LVM

pvs

vgs

lvs

Verify /data

df -h /data

mount | grep /data

Verify project permissions

ls -ld /project

ls -ld /project/docs

ls -ld /project/scripts

Expected:

drwxr-x---

Verify links

ls -li /project/docs

The hard link and original file should have the same inode number, while the symbolic link will point to the original path.

SECTION C — MINI CAPSTONE PROJECT

AlphaTech Server Recovery & Troubleshooting

Project Title

AlphaTech Server Recovery & Troubleshooting

Objective

The objective of this project is to troubleshoot and recover an AlphaTech Linux production server affected by multiple issues. The recovery includes SSH service failure, file permission problems, full disk storage, network connectivity failure and user login problems.

1. SSH Service Failing After Reboot

Problem

After rebooting the server, administrators are unable to connect through SSH.

Step 1: Check SSH service status

```
systemctl status sshd
```

If the service is stopped or failed, restart it.

Step 2: Restart SSH service

```
systemctl restart sshd
```

Check the status again:

```
systemctl status sshd
```

The expected status is:

Active: active (running)

Step 3: Enable SSH at boot

To make sure SSH automatically starts after every reboot:

```
systemctl enable sshd
```

Or both enable and start it:

```
systemctl enable --now sshd
```

Step 4: Verify SSH

```
systemctl is-enabled sshd
```

```
systemctl is-active sshd
```

Expected:

enabled

active

Result

The SSH service was restarted successfully and configured to start automatically after reboot.

2. Permission Denied on /project

Problem

A user receives Permission Denied while trying to access /project.

Step 1: Check ownership

```
ls -ld /project
```

Also check its subdirectories:

```
ls -ld /project/docs
```

```
ls -ld /project/scripts
```

Step 2: Check user and group

```
id dev1
```

```
id dev2
```

Check the development group:

```
getent group devteam
```

Step 3: Correct ownership

The required ownership is:

Owner = dev1

Group = devteam

Apply it:

```
chown -R dev1:devteam /project
```

Step 4: Correct permissions

The requirement is:

Owner = rwx

Group = r-x

Others = ---

This corresponds to permission 750.

```
chmod 750 /project
```

```
chmod 750 /project/docs
```

```
chmod 750 /project/scripts
```

Or:

```
chmod -R 750 /project
```

Step 5: Verify

```
ls -ld /project
```

```
ls -ld /project/docs
```

```
ls -ld /project/scripts
```

Expected:

```
drwxr-x---
```

Step 6: Test access

Switch to the required user:

```
su - dev1
```

Try:

```
cd /project
```

```
ls
```

Exit:

```
exit
```

Result

The ownership and permissions of /project were corrected and user access was successfully restored.

3. Disk is 100% Full — Extend LVM Storage

Problem

The server reports that the disk is almost or completely full.

Step 1: Check disk usage

`df -h`

Look for a filesystem showing approximately:

100%

For example:

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/mapper/alphavg-datalv	5G	5G	0	100%	/data

Step 2: Check LVM configuration

`pvs`

`vgs`

`lvs`

Check whether the Volume Group has free space.

For example:

VG	VSize	VFree
alphavg	10.00g	5.00g

If free space is available, the logical volume can be extended.

Step 3: Extend the Logical Volume

For example, add 2 GB:

`lvextend -L +2G /dev/alphavg/datalv`

You can also use:

`lvextend -r -L +2G /dev/alphavg/datalv`

The -r option also resizes the filesystem.

Step 4: If using XFS

If the LV was formatted with XFS and you did not use -r:

`xfs_growfs /data`

Step 5: If using ext4

For ext4:

`resize2fs /dev/alphavg/datalv`

Step 6: Verify storage

`df -h /data`

Also check:

`lvs`

`vgs`

Result

The logical volume was extended using available LVM space and the filesystem was expanded without requiring a server shutdown or reboot.

4. Server Cannot Reach the Internet

Problem

The server is unable to access the Internet.

The problem can be caused by an incorrect IP address, gateway, DNS configuration or network connection.

Step 1: Check IP address

`ip addr`

Check whether the network interface has the expected IP address.

Step 2: Check routing table

`ip route`

There should be a default route similar to:

default via 192.168.1.1 dev eth0

If the default gateway is missing, check the NetworkManager configuration.

Step 3: Check gateway connectivity

`ping -c 4 192.168.1.1`

If the gateway responds, the local network connection is working.

Step 4: Test Internet using IP address

`ping -c 4 8.8.8.8`

If this works:

The Internet connection is working and the problem is probably related to DNS.

If this fails:

Check the IP address, gateway, interface and network configuration.

Step 5: Check DNS

Check the DNS configuration:

```
cat /etc/resolv.conf
```

It should contain a valid DNS server, for example:

```
nameserver 8.8.8.8
```

You can also check NetworkManager DNS configuration:

```
nmcli dev show | grep DNS
```

Step 6: Test DNS resolution

```
ping -c 4 google.com
```

Or:

```
getent hosts google.com
```

If 8.8.8.8 works but google.com does not, the DNS configuration should be corrected.

Step 7: Correct network configuration

First identify the connection:

```
nmcli connection show
```

Example:

```
nmcli connection modify "System eth0" ipv4.gateway 192.168.1.1
```

```
nmcli connection modify "System eth0" ipv4.dns 8.8.8.8
```

```
nmcli connection up "System eth0"
```

Verify again:

```
ip addr
```

```
ip route
```

```
ping -c 4 8.8.8.8
```

```
ping -c 4 google.com
```

Result

The IP configuration, default gateway and DNS settings were checked and corrected. Internet connectivity was then verified using both an IP address and a domain name.

5. New Users Unable to Log In

Problem

Newly created users are unable to log in to the AlphaTech server.

Step 1: Verify user exists

```
id dev1
```

```
id dev2
```

If the users exist, their UID and groups will be displayed.

You can also check:

```
getent passwd dev1
```

```
getent passwd dev2
```

Step 2: Check password status

```
passwd -S dev1
```

```
passwd -S dev2
```

A normal account should not show the password as locked.

If required, set a new password:

```
passwd dev1
```

```
passwd dev2
```

Step 3: Check account expiration

```
chage -l dev1
```

```
chage -l dev2
```

Check that the account has not expired.

Step 4: Check group membership

```
id dev1
```

```
id dev2
```

The output should contain:

```
devteam
```

If the users are not members of the group:

```
usermod -aG devteam dev1
```

```
usermod -aG devteam dev2
```

Verify:

```
groups dev1
```

```
groups dev2
```

Step 5: Test login

Switch to the user locally:

```
su - dev1
```

If successful, the user account is working.

Exit:

```
exit
```

The same can be tested for dev2:

```
su - dev2
```

6. Complete Recovery Verification

After fixing all problems, perform the following final checks.

SSH

```
systemctl is-active sshd
```

```
systemctl is-enabled sshd
```

Expected:

```
active
```

```
enabled
```

Project permissions

```
ls -ld /project
```

```
ls -ld /project/docs
```

ls -ld /project/scripts

Expected:

drwxr-x---

Ownership

ls -ld /project

Expected:

dev1 devteam

Storage

df -h

pvs

vgs

lvs

Network

ip addr

ip route

ping -c 4 192.168.1.1

ping -c 4 8.8.8.8

ping -c 4 google.com

Users

id dev1

id dev2

passwd -S dev1

passwd -S dev2

groups dev1

groups dev2

7. Recovery Summary

Issue	Diagnosis	Recovery
SSH not working	systemctl status sshd	Restart and enable sshd
Permission denied	ls -ld /project	Correct chown and chmod

Issue	Diagnosis	Recovery
Disk 100% full	df -h, vgs, lvs	Extend LV and filesystem
Internet unavailable	ip addr, ip route, DNS checks	Correct IP, gateway and DNS
User cannot log in	id, passwd -S, chage, groups	Correct password/account/group settings

Final Conclusion

The AlphaTech production server was successfully diagnosed and recovered from multiple issues. The SSH service was restored and enabled at boot, project ownership and permissions were corrected, and the full filesystem was expanded using LVM. Network configuration was checked through IP, gateway and DNS tests, and user accounts were validated by checking their passwords, account status and group membership. The final verification confirmed that the major server services, storage, networking and user access were functioning correctly.

SECTION D — AI-AUGMENTED LEARNING

STEP 1 — BUILD WITH AI

Objective

The objective of this task is to use an AI tool to create a Bash script that automates basic Linux administration tasks such as user and group creation, permission configuration, disk usage checking and final status reporting.

Bash Script Generated with AI

Save the script as:

ai_server_setup.sh

Script:

```
#!/bin/bash
```

```
# User and group details
```

```
USERNAME="dev3"
```

```
GROUPNAME="devteam"
```

```
PROJECT_DIR="/project"
```

```
echo "Starting AlphaTech server setup..."
```

```
# Create group

groupadd "$GROUPNAME"

echo "[SUCCESS] Group $GROUPNAME created."


# Create user

useradd "$USERNAME"

echo "[SUCCESS] User $USERNAME created."


# Add user to group

usermod -aG "$GROUPNAME" "$USERNAME"

echo "[SUCCESS] User $USERNAME added to $GROUPNAME."


# Set password

echo "$USERNAME:Alpha@123" | chpasswd

echo "[SUCCESS] Password set for $USERNAME."


# Create project directory structure

mkdir -p "$PROJECT_DIR/docs"

mkdir -p "$PROJECT_DIR/scripts"

echo "[SUCCESS] Project directories created."


# Set ownership

chown -R "$USERNAME:$GROUPNAME" "$PROJECT_DIR"

echo "[SUCCESS] Project ownership configured."


# Set permissions

chmod 750 "$PROJECT_DIR"

chmod 750 "$PROJECT_DIR/docs"

chmod 750 "$PROJECT_DIR/scripts"

echo "[SUCCESS] Project permissions configured."
```

```
# Check disk usage
```

```
echo ""
```

```
echo "Current disk usage:"
```

```
df -h
```

```
# Final summary
```

```
echo ""
```

```
echo "===== SETUP SUMMARY ====="
```

```
echo "User created: $USERNAME"
```

```
echo "Group created: $GROUPNAME"
```

```
echo "User added to group: $GROUPNAME"
```

```
echo "Project directory: $PROJECT_DIR"
```

```
echo "Permissions: 750"
```

```
echo "Disk usage check: Completed"
```

```
echo "===== SETUP COMPLETED ====="
```

Give the script execute permission

```
chmod +x ai_server_setup.sh
```

Run the script

```
./ai_server_setup.sh
```

Expected output

Starting AlphaTech server setup...

[SUCCESS] Group devteam created.

[SUCCESS] User dev3 created.

[SUCCESS] User dev3 added to devteam.

[SUCCESS] Password set for dev3.

[SUCCESS] Project directories created.

[SUCCESS] Project ownership configured.

[SUCCESS] Project permissions configured.

Current disk usage:

Filesystem	Size	Used	Avail	Use%	Mounted on
...					

===== SETUP SUMMARY =====

User created: dev3

Group created: devteam

User added to group: devteam

Project directory: /project

Permissions: 750

Disk usage check: Completed

===== SETUP COMPLETED =====

STEP 2 — TEST & DEBUG WITHOUT AI

Testing the AI-generated script

The script was tested on the Linux server without using AI during the testing and debugging process.

During testing, one limitation was identified:

The script does not check whether the user or group already exists before creating them.

For example, if devteam already exists and the script is executed again:

```
groupadd devteam
```

may return:

```
groupadd: group 'devteam' already exists
```

Similarly:

```
useradd dev3
```

may fail if dev3 already exists.

This means the script is not safe to run multiple times.

Bug / Limitation Identified

Problem

The original script directly executes:

```
groupadd "$GROUPNAME"
```

```
useradd "$USERNAME"
```

without checking whether the group and user already exist.

Fix

I added checks using getent group and id.

Corrected Script

```
#!/bin/bash
```

```
USERNAME="dev3"
```

```
GROUPNAME="devteam"
```

```
PROJECT_DIR="/project"
```

```
echo "Starting AlphaTech server setup..."
```

```
# Check and create group
```

```
if getent group "$GROUPNAME" > /dev/null; then
```

```
    echo "[INFO] Group $GROUPNAME already exists."
```

```
else
```

```
    groupadd "$GROUPNAME"
```

```
if [ $? -eq 0 ]; then
```

```
    echo "[SUCCESS] Group $GROUPNAME created."
```

```
else
```

```
    echo "[ERROR] Failed to create group."
```

```
    exit 1
```

```
fi
```

```
fi
```

```
# Check and create user
```

```
if id "$USERNAME" > /dev/null 2>&1; then
    echo "[INFO] User $USERNAME already exists."
else
    useradd "$USERNAME"

    if [ $? -eq 0 ]; then
        echo "[SUCCESS] User $USERNAME created."
    else
        echo "[ERROR] Failed to create user."
        exit 1
    fi
fi

# Add user to group
usermod -aG "$GROUPNAME" "$USERNAME"

if [ $? -eq 0 ]; then
    echo "[SUCCESS] User $USERNAME added to $GROUPNAME."
else
    echo "[ERROR] Failed to add user to group."
    exit 1
fi

# Set password
echo "$USERNAME:Alpha@123" | chpasswd

if [ $? -eq 0 ]; then
    echo "[SUCCESS] Password configured for $USERNAME."
else
    echo "[ERROR] Failed to set password."
```

```
        exit 1
    fi

    # Create project directories
    mkdir -p "$PROJECT_DIR/docs" "$PROJECT_DIR/scripts"

    if [ $? -eq 0 ]; then
        echo "[SUCCESS] Project directories created."
    else
        echo "[ERROR] Failed to create project directories."
        exit 1
    fi

    # Set ownership
    chown -R "$USERNAME:$GROUPNAME" "$PROJECT_DIR"

    if [ $? -eq 0 ]; then
        echo "[SUCCESS] Project ownership configured."
    else
        echo "[ERROR] Failed to configure ownership."
        exit 1
    fi

    # Set permissions
    chmod 750 "$PROJECT_DIR"
    chmod 750 "$PROJECT_DIR/docs"
    chmod 750 "$PROJECT_DIR/scripts"

    if [ $? -eq 0 ]; then
        echo "[SUCCESS] Project permissions configured."
```

```

else

    echo "[ERROR] Failed to configure permissions."
    exit 1
fi

# Display disk usage
echo ""
echo "===== CURRENT DISK USAGE ====="
df -h
echo "===== "

# Final summary
echo ""
echo "===== SETUP SUMMARY ====="
echo "User: $USERNAME"
echo "Group: $GROUPNAME"
echo "Project: $PROJECT_DIR"
echo "Permissions: 750"
echo "Disk usage: Checked"
echo "Status: SETUP COMPLETED SUCCESSFULLY"
echo "===== "

Run the corrected script
chmod +x ai_server_setup.sh
./ai_server_setup.sh

Verification commands

After execution, I verified the configuration using:

id dev3

groups dev3

getent group devteam

ls -ld /project

```

`ls -ld /project/docs`

`ls -ld /project/scripts`

`df -h`

Expected permission

`drwxr-x---`

This represents:

Owner → `rwX`

Group → `r-x`

Others → `---`

which is permission 750, as required in Task 3.